Normalization

Chapter 5

Normalization

Normalization is the process to eliminate data redundancy and enhance data integrity in the table.

Normalization also helps to organize the data in the database.

It is a multi-step process that sets the data into tabular form and removes the duplicated data from the relational tables.

Normalization organizes the columns and tables of a database to ensure that database integrity constraints properly execute their dependencies.

It is a systematic technique of decomposing tables to eliminate data redundancy (repetition) and undesirable characteristics like Insertion, Update, and Deletion anomalies.

Data Redundancy

- Data redundancy in database means that some data fields are repeated in the database.
- Disadvantage
- ➢Increase database size and increase time
- ➢Inconsistency
- ➢Anamolies(Insertion,deletion,Updation)

Anomalies in DBMS

- There are three types of anomalies that occur when the database is not normalized. These are – Insertion, update and deletion anomaly. Let’s take an example to understand this.
- Example: If a table is not properly normalized and have data redundancy then it will not only eat up extra memory space but will also make it difficult to handle and update the database, without facing data loss. Insertion, Updation and Deletion Anomalies are very frequent if database is not normalized. To understand these anomalies let us take an example of a Student table.

Example

rollno	name	branch	hod	office_tel
1	A	CSE	Mr. X	12345
2	B	CSE	Mr. X	12345
3	C	CSE	Mr. X	12345
4	D	CSE	Mr. X	12345

In the table above, we have data of 4 Computer Sci. students. As we can see, data for the fields `branch`, `hod`(Head of Department) and `office_tel` is repeated for the students who are in the same branch in the college, this is **Data Redundancy**.

Insertion Anamoly

- When certain data cant be inserted onto database without the presence of other data.
- Suppose for a new admission, until and unless a student opts for a branch, data of the student cannot be inserted, or else we will have to set the branch information as NULL.
- Also, if we have to insert data of 100 students of same branch, then the branch information will be repeated for all those 100 students.
- These scenarios are nothing but Insertion anomalies.

Deletion Anamoly

- Deletion of some data causes the deletion of the other data.
- In our Student table, two different informations are kept together, Student information and Branch information. Hence, at the end of the academic year, if student records are deleted, we will also lose the branch information. This is Deletion anomaly.

Update Anamoly

- When we want to update a single piece of data but it must be done for all other data.
- What if Mr. X leaves the college? or is no longer the HOD of computer science department? In that case all the student records will have to be updated, and if by mistake we miss any record, it will lead to data inconsistency. This is Updation anomaly.

Functional Dependency

A Functional Dependency in DBMS is a fundamental concept that describes the relationship between attributes (columns) in a table.

It shows how the values in one or more attributes determine the value in another.

In layperson's terms, it describes how data in one column or set of columns can relate to data in another column. It helps to maintain the quality of the data in DBMS.

Functional Dependency is represented in the form of an equation.

Here, you have a set of attributes (A, B, C, etc.) and an arrow (->) denoting the Dependency.

For example, if we have a table of employee data with columns "EmployeeID," "FirstName," and "LastName," we can express a functional dependency like this: **EmployeeID -> FirstName, LastName.**

How to denote a functional dependency in DBMS

In DBMS, you denote functional dependencies using a notation. It contains two main components: the left-hand side (LHS) and the right-hand side (RHS) of an arrow (->).

For example, if we have a table with attributes "A," "B," and "C," and attribute "A" determines the values of attributes "B" and "C," you would denote it as

A -> B, C

This notation indicates that the value(s) in attribute "A" determines the value(s) in attributes "B" and "C." In other words, if you know the value of "A," you can determine the values of "B" and "C."

Types of Functional Dependencies

- Trivial functional dependency
- non-trivial functional dependency
- Multivalued dependency
- Transitive dependency

Trivial Functional Dependency

A trivial functional dependency in DBMS occurs when an attribute or set of attributes (columns) on the left-hand side (LHS) of a functional dependency arrow (->) already determines the attributes on the right-hand side (RHS) without any extra information.

Suppose we have a table of students with attributes "StudentID" and "StudentName." In this case, if we state the functional dependency as

StudentID -> StudentName.

This is a trivial functional dependency. Because within a single "StudentID," there can be only one corresponding "StudentName." In other words, the value of "StudentID" determines the value of "StudentName" without any more information or conditions.

Non-trivial Functional Dependency

A non-trivial functional dependency is a specific type of dependency between attributes (columns) in a table.

Here, the relationship is not obvious or trivial.

It conveys meaningful information about how the values in one set of attributes determine the values in another.

Example:

Student Id	Student Name	Student DOB	Class
101	Alice	1995-05-15	10A
102	Bob	2000-03-20	10B
103	Carol	1999-07-10	10A
104	Dave	2000-01-05	10B

We want to express a functional dependency based on the student's birthdate (StudentDOB) and class (Class).
A non-trivial functional dependency in this case would be

StudentDOB, Class -> StudentName

This functional dependency means that given a combination of a student's date of birth and class, you can uniquely determine their name.
It's non-trivial because it provides valuable information about the relationship between attributes in the table.

Multi-valued Dependency

- Multivalued dependency occurs when there are more than one independent multivalued attributes in a table.
- A multi-valued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

Example:

bike_model	manuf_year	color
M1001	2007	Black
M1001	2007	Red
M2012	2008	Black
M2012	2008	Red
M2222	2009	Black
M2222	2009	Red

Here columns manuf_year and color are independent of each other and dependent on bike_model.
In this case these two columns are said to be multivalued dependent on bike_model.
These dependencies can be represented like this:
bike_model ->> manuf_year
bike_model ->> color

Transitive Dependency

A transitive functional dependency in DBMS is a relational database table's relationship between attributes (columns).

It occurs when one attribute's value determines another's value through an intermediary (a third) attribute.

$X \rightarrow Z$ is a transitive dependency if the following three functional dependencies hold true:

- $X \rightarrow Y$
- Y does not $\rightarrow X$
- $Y \rightarrow Z$

Note: A transitive dependency can only occur in a relation of three or more attributes. This dependency helps us normalizing the database in 3NF (3rd Normal Form).

Example

Consider a database table called "Student_Info" with the following attributes:

Student_ID (unique identifier for each student),

Student_Name,

Student_Address

Student_City.

In this example, we can assume that Student_Address depends on Student_City, and Student_City depends on Student_ID.

This creates a transitive dependency in DBMS, where Student_ID indirectly determines Student_Address.

Another Example:

Book	Author	Author_age
Game of Thrones	George R. R. Martin	66
Harry Potter	J. K. Rowling	49
Dying of the Light	George R. R. Martin	66

{Book} →{Author} (if we know the book, we knows the author name)
{Author} does not →{Book}
{Author} → {Author_age}
Therefore as per the rule of **transitive dependency**: {Book} → {Author_age} should hold, that makes sense because if we know the book name we can know the author's age.

First Normal Form (1NF)

A table is referred to as being in its First Normal Form if atomicity of the table is 1.

Here, atomicity states that a single cell cannot hold multiple values.

It must hold only a single-valued attribute.

The First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Example of 1NF

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	9806473823 9845009823	Kathmandu
20	Harry	9845478383	Pokhara
12	Sam	9830302850 9865903783	Lalitpur

The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_STATE
14	John	Kathmandu
14	John	Kathmandu
20	Harry	Pokhara
12	Sam	Lalitpur
12	Sam	Lalitpur

EMP_ID	EMP_PHONE
14	9806473823
14	9845009823
20	9845478383
12	9830302850
12	9865903783

Partial Dependency

Partial Dependency occurs when a non-prime attribute is functionally dependent on part of a candidate key.

The 2nd Normal Form (2NF) eliminates the Partial Dependency.

<StudentProject>

StudentID	ProjectNo	StudentName	ProjectName
S01	199	Katie	Geo Location
S02	120	Ollie	Cluster Exploration

In the above table, we have partial dependency; let us see how –

The prime key attributes are **StudentID** and **ProjectNo**, and

StudentID = Unique ID of the student
StudentName = Name of the student
ProjectNo = Unique ID of the project
ProjectName = Name of the project

As stated, the non-prime attributes i.e. **StudentName** and **ProjectName** should be functionally dependent on part of a candidate key, to be Partial Dependent.

The **StudentName** can be determined by **StudentID**, which makes the relation Partial Dependent.

The **ProjectName** can be determined by **ProjectNo**, which makes the relation Partial Dependent.

Second Normal Form (2NF)

The first condition for the table to be in Second Normal Form is that the table has to be in First Normal Form.

The table should not possess partial dependency.

The partial dependency here means the proper subset of the candidate key should give a non-prime attribute.

For the above example

As stated, the non-prime attributes i.e. **StudentName** and **ProjectName** should be functionally dependent on part of a candidate key, to be Partial Dependent.

The **StudentName** can be determined by **StudentID**, which makes the relation Partial Dependent.

The **ProjectName** can be determined by **ProjectNo**, which makes the relation Partial Dependent.

Therefore, the <StudentProject> relation violates the 2NF in Normalization and is considered a bad database design.

To remove Partial Dependency and violation on 2NF, decompose the tables –

<StudentInfo>

StudentID	ProjectNo	StudentName
S01	199	Katie
S02	120	Ollie

<ProjectInfo>

ProjectNo	ProjectName
199	Geo Location
120	Cluster Exploration

Third Normal Form

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

Example

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	44800	Bhaktapur	Bhaktapur
333	Stephan	21003	Humla	Muchu
444	Lan	57302	Ilam	Nayabazar
555	Katharine	57204	Jhapa	Birtamod
666	John	21204	Jumla	Tatopani

- In the given table, all attributes except EMP_ID are non-prime.
- Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.
- That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	44800
333	Stephan	21003
444	Lan	57302
555	Katharine	57204
666	John	21204

EMP_ZIP	EMP_STATE	EMP_CITY
44800	Bhaktapur	Bhaktapur
21003	Humla	Muchu
57302	Ilam	Nayabazar
57204	Jhapa	Birtamod
21204	Jumla	Tatopani

Question

Normalize the table into 3NF

Tournament	Year	Winner	Winner_DOB
Indiana Invitational	1998	All Fredrickson	1975/5/9
Cleveland	1999	Bob Albertson	1968/9/22
Des Moines	1999	All Fredrickson	1975/5/9
Indiana Invitational	1998	Chip Masterson	1977/7/13

In this table, following transitive functional dependencies are existing:

(Tournment, Year) -> Winner

Winner -> Winner_DOB

Here, transitive dependency is presence.

Boyce Codd Normal Form (BCNF)

Boyce Codd Normal Form is also known as 3.5 NF.

It is the superior version of 3NF and was developed by Raymond F. Boyce and Edgar F. Codd to tackle certain types of anomalies which were not resolved with 3NF.

The first condition for the table to be in Boyce Codd Normal Form is that the table should be in the third normal form.

Secondly, every Right-Hand Side (RHS) attribute of the functional dependencies should depend on the super key of that particular table.

For example :

You have a functional dependency X → Y. In the particular functional dependency, X has to be the part of the super key of the provided table.

Consider the given table

stuid	subject	professor
1	SQL	Prof. Mishra
2	Java	Prof. Anand
2	C++	Prof. Kanth
3	Java	Prof. James
4	DBMS	Prof. Lokesh

The subject table follows these conditions:

- Each student can enroll in multiple subjects.
- Multiple professors can teach a particular subject.
- For each subject, it assigns a professor to the student.

In the above table, student_id and subject together form the primary key because using student_id and subject; you can determine all the table columns.

Another important point to be noted here is that one professor teaches only one subject, but one subject may have two professors.

Which exhibit there is a dependency between subject and professor, i.e. subject depends on the professor's name.

The table is in 1st Normal form as all the column names are unique, all values are atomic, and all the values stored in a particular column are of the same domain.

The table also satisfies the 2nd Normal Form, as there is no Partial Dependency.

And, there is no Transitive Dependency; hence, the table also satisfies the 3rd Normal Form.

This table follows all the Normal forms except the Boyce Codd Normal Form.

As you can see stuid, and subject forms the primary key, which means the subject attribute is a prime attribute.

However, there exists yet another dependency - professor → subject.

BCNF does not follow in the table as a subject is a prime attribute, the professor is a non-prime attribute.

To transform the table into the BCNF, you will divide the table into two parts. One table will hold stuid which already exists and the second table will hold a newly created column profid.

stuid	profid
1	101
2	102
2	103
3	102
4	104

profid	subject	professor
1	SQL	Prof. Mishra
2	Java	Prof. Anand
2	C++	Prof. Kanth
3	Java	Prof. James
4	DBMS	Prof. Lokesh

Fourth Normal Form (4NF)

- The database must satisfy the following two things;
  - The database must meet all the requirement of 3NF
  - The table should not have any **Multi-valued Dependency**.

- According to the 4th normal form, a record type should not contain two or more independent multi-valued facts about an entity.

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

- In the table above, there is no relationship between the columns course and hobby. They are independent of each other.
- And both are depends on same attribute STU_ID.
- So there is multi-valued dependency, which leads to unnecessary repetition of data and other anomalies as well.

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

Fifth Normal Form (5NF)

- Table should be in 4NF.
- If we can decompose table further to eliminate redundancy and anomaly, and when we re-join the decomposed tables by means of candidate keys, we should not be losing the original data or any new record set should not arise.

(In simple words, joining two or more decomposed table should not lose records nor create new records.)

OR, Eliminate join dependency.

Join Functional Dependency

- If a table can be recreated by joining multiple tables and each of this table have a subset of the attributes of the table, then the table is in Join Dependency.
- It is a generalization of Multi-valued Dependency.

Example

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

P1

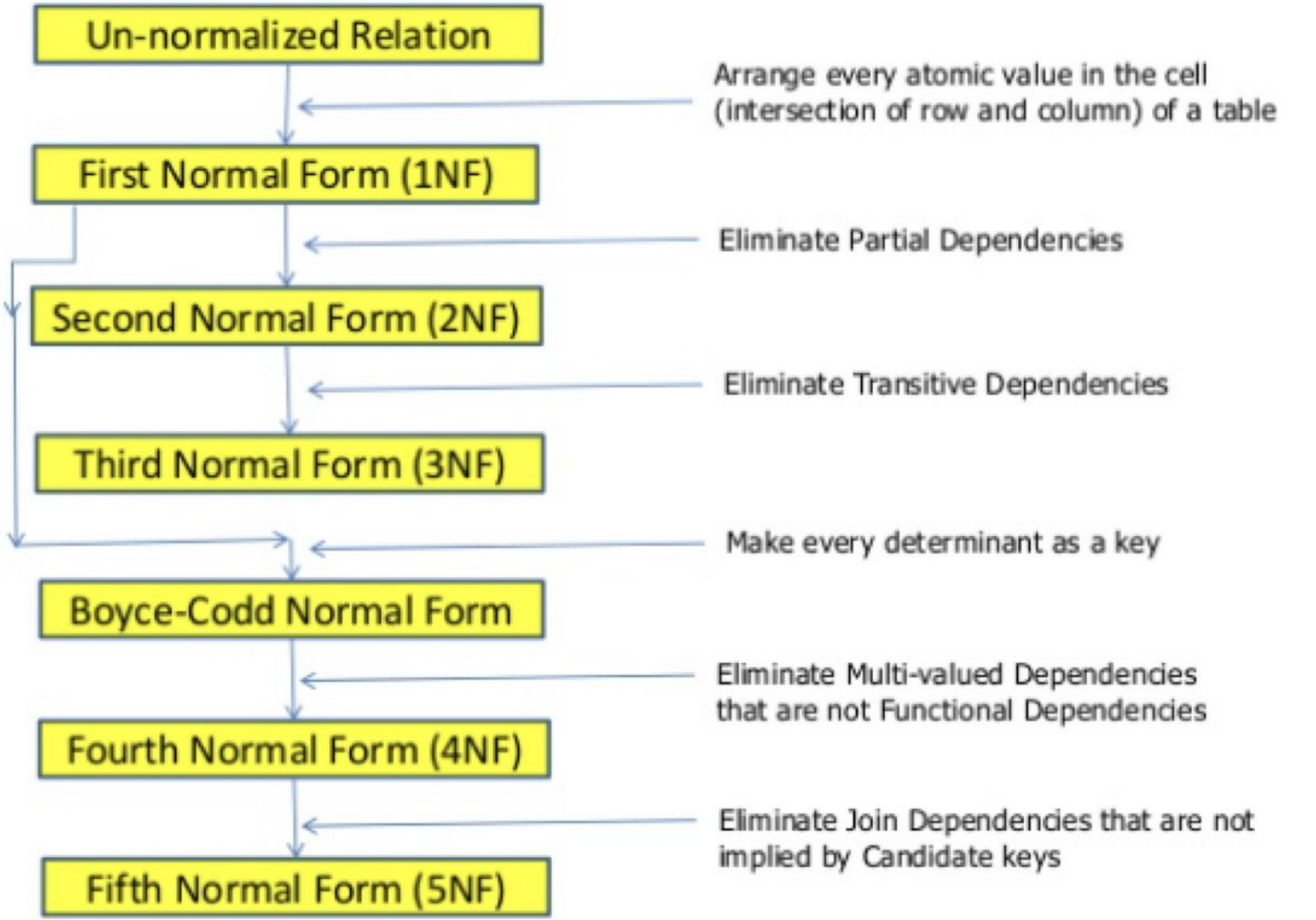
SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

P2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

SEMSTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen

The decompositions P1,P2,P3 for a relation schema R are said to be Lossless if there natural join results the original relation R. The decompositions P1,P2 and P3 for a relation schema R are said to be Lossy if there natural join results into addition of extraneous tuples with the original relation R

Summary of all normal forms